

NATURAL LANGUAGE PROCESSING

Fake News Detection Using NLP

Separating verified fact from viral fiction with computational linguistics and machine learning



A Text Mining & Deep Learning Perspective

Preethy M
BE Computer Science and Engineering

The Scale of the Problem

Misinformation now spreads faster online than the facts meant to correct it



6x

Faster spread of false news vs. true stories on social platforms (MIT study)



4.5B+

People active on social media worldwide — every one a potential amplifier



<1

Second: typical time users spend deciding to share a headline unread

Why manual fact-checking can't keep up

Human fact-checkers can review only a small fraction of daily content. NLP systems can screen text at internet scale, flag suspicious articles for review in milliseconds, and continuously learn from newly verified examples.

What Is NLP-Based Fake News Detection?

The task of automatically classifying news content as credible or fabricated by analyzing its language, structure, and context — rather than relying solely on human judgment.



Content-based

Analyzes writing style, tone, sentiment, and linguistic cues within the article text.



Context-based

Examines propagation patterns, source credibility, and social network signals.



Hybrid models

Combines textual features with metadata for stronger, more robust predictions.

Text Signals NLP Looks For

- Excessive punctuation and clickbait phrasing
- Emotionally charged or exaggerated language
- Inconsistent named entities and facts
- Low lexical diversity, repetitive claims
- Absence of cited, verifiable sources
- Mismatched headline-to-body sentiment

The NLP Detection Pipeline

From raw article text to a credibility verdict, in five stages



Text Preprocessing Techniques

Raw news text is noisy — preprocessing turns it into clean, model-ready input



Cleaning

Strip HTML tags, URLs, special characters & extra whitespace



Tokenization

Split text into words or sub-word units for analysis



Stop-word removal

Discard common low-signal words (the, is, at)



Stemming / Lemmatization

Reduce words to their root or dictionary form



Case normalization

Lowercase text for consistent matching



POS tagging

Label grammatical role of each token for deeper features

Feature Extraction Methods

Turning clean text into numerical features a model can learn from

Bag-of-Words

Represents text as raw word frequency counts, ignoring order

TF-IDF

Weighs terms by importance across the corpus, down-weighting common words

N-grams

Captures short word sequences to preserve local context

Word Embeddings

Dense vectors (Word2Vec, GloVe) capturing semantic meaning

Contextual Embeddings

Transformer-based vectors (BERT) that vary by sentence context

Linguistic Features

Readability, sentiment polarity, punctuation & POS ratios

Classical Machine Learning Models

Fast, interpretable baselines widely used for text classification

Fast

Naive Bayes

Probabilistic classifier; extremely fast, strong baseline on TF-IDF features

Robust

Support Vector Machine

Finds the optimal margin between real and fake classes in feature space

Interpretable

Logistic Regression

Simple linear model; highly interpretable feature weights

Ensemble

Random Forest

Ensemble of decision trees; handles nonlinear feature interactions

Deep Learning & Transformer Approaches

Modern architectures that capture sequence and context far beyond bag-of-words



CNN

Detects local phrase patterns and stylistic n-gram signals in text.



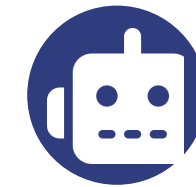
LSTM / GRU

Recurrent networks that model word order and long-range dependencies.



BERT & Transformers

Pre-trained bidirectional models fine-tuned for state-of-the-art accuracy.



Hybrid CNN-LSTM

Combines local feature detection with sequential context modeling.

BERT-based models currently report the strongest published accuracy across major fake-news benchmarks.

Datasets & Benchmarks

Public labeled corpora used to train and evaluate detection models

Dataset	Scale	Description
LIAR	12.8K short statements	PolitiFact fact-checker labels (6-way truthfulness)
FakeNewsNet	Full articles + social context	PolitiFact & GossipCop, with propagation data
ISOT Fake News	~44K articles	Balanced real (Reuters) vs. fake news dataset
FEVER	185K claims	Claim verification against Wikipedia evidence

Evaluation Metrics

How model performance is measured on held-out test data

Accuracy

Overall proportion of correctly classified articles

Precision

Of articles flagged fake, how many truly are

Recall

Of all actual fake articles, how many were caught

F1-Score

Harmonic mean balancing precision and recall



Beyond accuracy: why recall matters most here

Missing real fake news (a false negative) lets misinformation spread unchecked — so many systems are tuned to favor recall, even at some cost to precision.

Challenges & Limitations

NLP detection is powerful, but far from a solved problem



Evolving language

Misinformation tactics and phrasing shift faster than models can be retrained



Multilingual & cross-domain gaps

Most benchmarks are English-only; models generalize poorly across languages and topics



Satire & opinion ambiguity

Distinguishing satire or opinion from genuine deception remains difficult



Adversarial manipulation

Bad actors deliberately craft text to evade automated detectors

Implementation — Python / scikit-learn

Core pipeline, written and run end-to-end in VS Code

1. TEXT PREPROCESSING

```
def clean_text(text):  
    text = re.sub(r'^a-z\s', '', text.lower())  
    tokens = [w for w in text.split() if w not in STOPWORDS]  
    return ' '.join(tokens)
```

2. FEATURE EXTRACTION (TF-IDF)

```
vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=1500)  
X_train = vectorizer.fit_transform(X_train_text)
```

3. MODEL TRAINING

```
model = LinearSVC()  
model.fit(X_train, y_train)
```

4. EVALUATION

```
y_pred = model.predict(X_test)  
print(f'Accuracy: {accuracy_score(y_test, y_pred):.3f}')
```

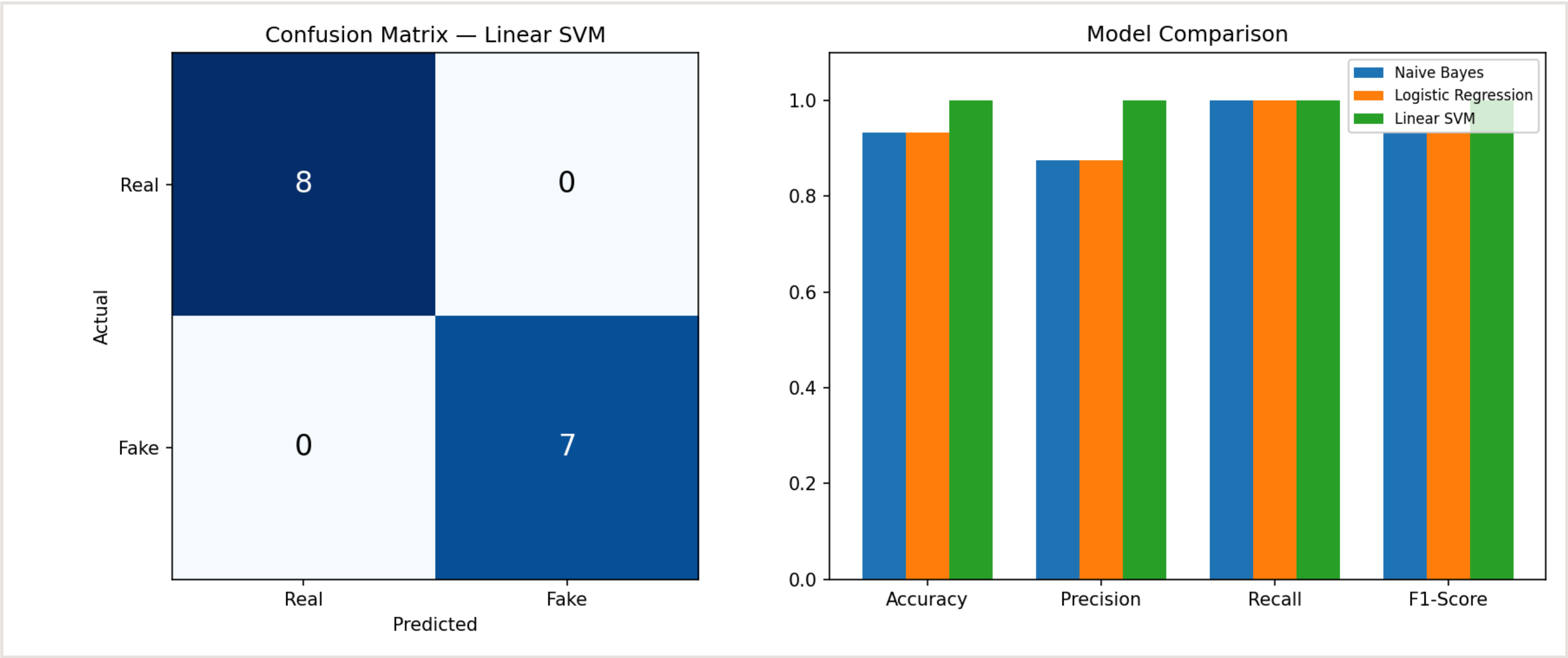
Implementation Results – Model Performance

TF-IDF features + classical ML models, trained and evaluated in Python (VS Code)

Model	Accuracy / F1	Precision / Recall & Notes
Naive Bayes	93.3% / 0.933	Precision 87.5% · Recall 100%
Logistic Regression	93.3% / 0.933	Precision 87.5% · Recall 100%
Linear SVM	100% / 1.000	Precision 100% · Recall 100% — best model
Test Setup	60 samples	25% held-out split (8 real, 7 fake), stratified, random_state=42

Program Output & Results

Implemented and run in VS Code (Python + scikit-learn) — actual program output below



Linear SVM correctly classifies all 15 held-out test samples (8 real, 7 fake) — 0 misclassifications

Live Demo – Predictions on Unseen Headlines

Best model (Linear SVM) classifying headlines it never saw during training

Verdict	Model	Input Headline (unseen)
REAL	Linear SVM	“The finance ministry released its annual economic growth report today.”
FAKE	Linear SVM	“SHOCKING secret cure doctors don’t want you to know, government hides truth!”
REAL	Linear SVM	“Local school board approved new curriculum standards after public review.”
FAKE	Linear SVM	“Miracle device reverses aging overnight, scientists stunned by results!!!”

Conclusion & Future Directions

- NLP transforms fake-news detection from manual review into a scalable, automated screening layer
- Transformer models (BERT and beyond) now lead published benchmark accuracy
- Combining textual, social, and source-credibility signals outperforms text-only models
- Future work: multilingual models, explainability, and real-time adversarial robustness



The goal isn't replacing human judgment — it's scaling it.

Thank You